UNIVERSITÉ SAINT JEAN
SAINT JEAN INGÉNIEUR

UNIVERSITÉ SAINT JEAN INGÉNIEUR

RAPPORT FINAL DE PROJET

Conteneurisation et Gestion des Workloadsavec OpenStack Magnum & Kubernetes

Rédigé par :

ESSONO EFFA Etienne Florian

NGNINTEDEM FEUPI Vaillant

Sous la supervision de :

M. NGUIMBUS Emmanuel

Année Académique 2025 – 2026

Résumé Exécutif

Ce rapport présente la réalisation complète du projet de conteneurisation et de gestion des workloads avec OpenStack Magnum et Kubernetes, mené dans le cadre de la formation d'ingénieur à l'Université Saint Jean pour l'année académique 2025–2026.

Face aux défis croissants des architectures cloud-natives — conteneurisation, microservices, orchestration hybride — ce projet a consisté à déployer une plateforme OpenStack complète via Kolla-Ansible sur Ubuntu 24.04 LTS, permettant la gestion automatisée de clusters Kubernetes via Magnum et l'orchestration d'infrastructures hybrides (VM + conteneurs) via Heat.

Le projet couvre l'intégralité du cycle de vie : de l'installation de l'environnement OpenStack jusqu'à la création de clusters Kubernetes, en passant par la définition de templates HOT, les tests de montée en charge et la mise en place du monitoring. Toutes les erreurs rencontrées en cours de déploiement ont été documentées et résolues, constituant un guide de dépannage précieux pour de futurs déploiements similaires.

Points clés : Déploiement OpenStack via Kolla-Ansible sur VirtualBox Ubuntu 24.04 — Clusters Kubernetes gérés par Magnum — Stacks hybrides VM+K8s via Heat — Documentation complète des erreurs et solutions.

Table des Matières

1. Introduction et Contexte du Projet
2. Cahier des Charges
    2.1 Contexte et Justification
    2.2 Cibles et Bénéficiaires
    2.3 Objectifs du Projet
    2.4 Expression des Besoins
    2.5 Ressources Utilisées
    2.6 Planification
3. Cahier d'Analyse
    3.1 Modélisation UML — Acteurs et Rôles
    3.2 Diagramme de Cas d'Utilisation
    3.3 Description Textuelle des Cas d'Utilisation
4. Cahier de Réalisation
    4.1 Environnement Logiciel et Technologies
    4.2 Architecture du Système
5. Architecture Réseau et Services OpenStack
6. Guide de Déploiement Complet
    6.1 Prérequis Matériels
    6.2 Séquence d'Installation Kolla-Ansible
    6.3 Configuration Post-Déploiement
    6.4 Déploiement via Magnum (Kubernetes)
    6.5 Orchestration Hybride via Heat
7. Problèmes Rencontrés et Solutions
8. Gestion, Sécurité et Bonnes Pratiques
9. Conclusion et Perspectives

Chapitre 1 — Introduction et Contexte du Projet

1.1 Contexte Général

L'évolution rapide des architectures applicatives, désormais fortement orientées vers le cloud, les microservices et les conteneurs, impose aux entreprises de disposer de plateformes capables de gérer efficacement des workloads distribués et dynamiques. Les organisations doivent non seulement accélérer le déploiement des applications, mais également garantir la scalabilité, la résilience, la flexibilité et l'automatisation de l'infrastructure sous-jacente.

Dans un environnement multi-locataire ou hybride, la gestion manuelle de machines virtuelles, réseaux, clusters et applications devient complexe, coûteuse et source d'erreurs. Pour répondre à ces enjeux, OpenStack s'est imposé comme une solution cloud open source complète, et ses services Magnum (gestion des clusters de conteneurs) et Heat (orchestration d'infrastructure) permettent d'industrialiser, standardiser et automatiser le déploiement d'environnements conteneurisés modernes tels que Kubernetes et Docker Swarm.

1.2 Problématique

Ce projet s'inscrit dans un contexte où la conteneurisation est devenue un standard technologique incontournable, et où les entreprises recherchent des solutions leur permettant d'unifier la gestion de leurs infrastructures VM et conteneurs, tout en optimisant l'efficacité opérationnelle et les coûts.

> Pourquoi Kolla-Ansible et non DevStack ? DevStack sur Ubuntu 24.04 génère des incompatibilités critiques : blocage de init_placement après 4h (OOM Killer sans swap), conflit OVN vs q-agt, et incompatibilités Python 3.12. Kolla-Ansible déploie chaque service OpenStack dans un conteneur Docker isolé, éliminant tous ces conflits et offrant une solution recommandée en production par la communauté OpenStack.

1.3 Enjeux Stratégiques

- **Transition vers une architecture cloud-native**
- **Réduction de 80% du temps de déploiement manuel**
- **Amélioration de 40% de l'utilisation des ressources**
- **Capacité à déployer des applications en moins de 15 minutes**

Chapitre 2 — Analyse des besoins fonctionnels et non fonctionnels

2.1 Contexte et Justification

Contexte Actuel

- Croissance exponentielle des workloads conteneurisés dans les environnements cloud modernes
- Complexité accrue de la gestion des infrastructures hybrides (VM + conteneurs)
- Besoin urgent d'automatisation des déploiements et de la gestion des clusters
- Défis de sécurité et d'isolation dans les environnements multi-locataires

Justification du Projet

- Réduction des coûts via l'automatisation et l'optimisation des ressources cloud
- Amélioration de la productivité des équipes DevOps grâce à des outils standardisés
- Flexibilité accrue pour le déploiement d'applications modernes et microservices
- Optimisation de l'utilisation des ressources avec scaling dynamique

2.2 Cibles et Bénéficiaires

Publics Cibles Principaux

- Administrateurs systèmes et cloud responsables de l'infrastructure
- Ingénieurs DevOps et SRE en charge de l'automatisation et des déploiements
- Équipes de développement d'applications conteneurisées
- Responsables d'infrastructure informatique en entreprise

Bénéficiaires Finaux

- Services informatiques des entreprises souhaitant moderniser leur infrastructure
- Prestataires de services cloud proposant des offres OpenStack
- Institutions académiques et de recherche — dont l'Université Saint Jean
- Startups et PME technologiques nécessitant une plateforme cloud privée agile

2.3 Objectifs du Projet

Objectif Général

Mettre en place une solution permettant de déployer, orchestrer et automatiser la gestion de clusters Kubernetes ou Docker Swarm via le service Magnum, tout en intégrant des stacks hybrides (VM + conteneurs) grâce au service Heat, dans un environnement OpenStack complet déployé via Kolla-Ansible.

Objectifs Spécifiques

1. Comprendre l'architecture OpenStack et identifier le rôle de chaque service impliqué (Nova, Neutron, Heat, Keystone, Magnum, Cinder, Glance).

2. Déployer des clusters Kubernetes automatiquement via Magnum en s'appuyant sur des ClusterTemplates standardisés ou personnalisés.
3. Intégrer Magnum avec Heat pour créer des stacks hybrides combinant VMs, réseaux, stockage et clusters de conteneurs dans une description déclarative unique.
4. Automatiser le cycle de vie des clusters depuis la création jusqu'à la mise à l'échelle, le monitoring, la sécurisation et la suppression.
5. Mettre en place un système de monitoring (Prometheus, Grafana ou équivalent) pour surveiller CPU, mémoire, nœuds et pods.
6. Implémenter des mécanismes de scaling automatique pour optimiser l'utilisation des ressources.
7. Documenter et livrer scripts, playbooks, templates HOT, diagrammes UML et rapport complet.

2.4 Indicateurs Clés de Performance

Indicateur	Valeur Cible	Impact	Statut
Temps de déploiement cluster	< 15 minutes	Haute	Défini
Disponibilité des services	99%	Critique	Défini
Clusters supportés simultanément	10+	Moyenne	Défini
Temps de réponse API	< 2 secondes	Haute	Défini
ROI attendu	12–18 mois	Critique	Estimé

2.5 Expression des Besoins

Les Services OpenStack Utilisés

OpenStack est une plateforme de cloud computing open source permettant de gérer des pools de ressources de calcul, de stockage et de réseau au sein d'un centre de données. Voici les services mobilisés dans ce projet :

Service	Nom de code	Rôle Principal	Analogie AWS
Compute	Nova	Gestion des machines virtuelles (KVM)	Amazon EC2
Networking	Neutron	Réseaux virtuels, SDN, OVN	Amazon VPC
Block Storage	Cinder	Volumes persistants attachables aux VMs	Amazon EBS
Object Storage	Swift	Stockage d'objets à grande échelle	Amazon S3
Image Service	Glance	Catalogue d'images OS	Amazon AMI
Identity	Keystone	Authentification et autorisation (RBAC)	AWS IAM
Dashboard	Horizon	Interface web d'administration	AWS Console
Orchestration	Heat	Infrastructure as Code via templates HOT	AWS CloudFormation
COE Manager	Magnum	Gestion de clusters Kubernetes/Swarm	Amazon EKS
Placement	Placement	Allocation des ressources CPU/RAM	AWS Scheduler

Interaction Clé : Magnum ↔ Heat

Dans ce projet, Magnum ne crée pas directement de serveurs. Lorsqu'un cluster est demandé via Magnum, celui-ci génère en arrière-plan un template Heat. Cette intégration est vitale pour deux raisons :

- **Heat permet d'écrire un template unique qui définit simultanément des instances Nova (VMs pour bases de données ou applications legacy) et un cluster Magnum (pour les microservices). Les deux mondes partagent le même réseau privé et les mêmes règles de sécurité dès le déploiement.**
- **En utilisant Heat comme moteur, Magnum bénéficie de la robustesse d'un outil éprouvé pour la gestion des dépendances entre ressources (ex : ne pas lancer le cluster avant que le réseau soit prêt).**

Besoins Fonctionnels

- **Création, modification, suppression via API et CLI**
- **Gestion combinée VMs et conteneurs dans des stacks unifiées**
- **Supervision des performances et de la santé des clusters**
- **Backup des configurations et données critiques**
- **Ajustement automatique des ressources (scale-up / scale-down)**
- **Management via Horizon (web) et CLI OpenStack**

Besoins Non Fonctionnels

- **Temps de déploiement d'un cluster < 15 minutes**
- **99% pour les services critiques**
- **Isolation réseau, authentification forte (Keystone), chiffrement TLS**
- **Support jusqu'à 100 nœuds par cluster**
- **Architecture modulaire, Infrastructure as Code versionnée sous Git**
- **Compatibilité avec les standards OpenStack et Kubernetes**

2.6 Ressources Utilisées

Ressources Humaines

Rôle	Nb. Jours	Taux/Jour (FCFA)	Coût Total (FCFA)
Architecte Cloud	10 jours	300 000	3 000 000
Administrateur OpenStack	50 jours	250 000	12 500 000
Ingénieur DevOps	50 jours	250 000	12 500 000
Expert Sécurité	5 jours	350 000	1 750 000
Chef de Projet	15 jours	250 000	3 750 000
TOTAL	130 jours	Moy. 280 000	**33 500 000**

Budget Consolidé

Poste Budgétaire	Montant (FCFA)	Pourcentage
Ressources Humaines (Architecte, Admin, DevOps, Sécurité, Chef de projet)	33 500 000	82,13%
Infrastructure Cloud (Serveurs, Stockage, Réseau, Backup)	1 965 000	4,81%
Imprévus (15%)	5 319 750	13,04%
TOTAL GÉNÉRAL	**40 784 750**	**100%**

2.7 Planification

Phase	Durée	Période	Livrables Principaux
Phase 1	2 semaines	J1 – J14	Environnement OpenStack opérationnel, VM configurée
Phase 2	4 semaines	J15 – J42	Clusters K8s/Swarm déployables via Magnum
Phase 3	2 semaines	J43 – J56	Intégration Magnum-Heat fonctionnelle, stacks hybrides
Phase 4	2 semaines	J57 – J70	Solution complète validée, rapport final livré

Chapitre 3 —Analyse et Modélisation UML

3.1 Acteurs et Rôles

Acteur	Rôle
Administrateur Cloud (Effa & Feupi)	Configure OpenStack, gère les projets, quotas, réseaux, clés SSH et politiques de sécurité. Valide les templates et supervise l'ensemble du déploiement.
Ingénieur DevOps (Ngnintedem Feupi)	Automatise les déploiements, crée et gère les clusters, intègre les pipelines CI/CD, effectue le scaling et configure le monitoring.
Développeur d'Applications (Effa Etienne)	Déploie des applications conteneurisées sur les clusters Kubernetes/Swarm fournis par la plateforme.
Service Magnum	Service OpenStack responsable de la gestion complète du cycle de vie des clusters COE (création, scaling, suppression).
Service Heat	Moteur d'orchestration des stacks hybrides VM + clusters. Interprète et exécute les templates HOT en YAML.
API OpenStack	Point d'entrée unifié pour toutes les interactions programmatiques avec la plateforme (CLI, scripts, Ansible, Terraform).

3.2 Cas d'Utilisation

Le système MAGSYS_HEAT couvre six cas d'utilisation principaux, regroupés autour des acteurs identifiés. Le diagramme de cas d'utilisation met en évidence les interactions entre l'Administrateur Cloud, l'Ingénieur DevOps, le Développeur et l'API OpenStack, avec les fonctionnalités clés : Créer un cluster, Déployer une stack hybride, Mettre à l'échelle, Automatiser via script, Supprimer un cluster, et Récupérer les credentials (kubeconfig).

3.3 Description Textuelle des Cas d'Utilisation

UC01 — Créer un Cluster Kubernetes via Magnum

Titre	UC01 — Créer un Cluster Kubernetes
Résumé	Permet à un administrateur ou ingénieur DevOps de créer un cluster Kubernetes en utilisant un ClusterTemplate Magnum.
Acteurs	Administrateur Cloud, Ingénieur DevOps, Service Magnum
Précondition	OpenStack fonctionnel, ClusterTemplate configuré, quotas disponibles, réseau accessible.
Scénario	1. Authentification sur Keystone via Horizon ou CLI.2. Consultation et sélection du ClusterTemplate.3. Commande de création avec paramètres (nom, template, master-count, node-count).4. Magnum valide les droits et génère un template Heat HOT.5. Heat crée les ressources : réseau (Neutron), volumes (Cinder), VMs (Nova).6. Nova lance les instances Fedora CoreOS avec cloud-init pour installer Kubernetes.7. Magnum surveille l'état — statut CREATE_COMPLETE quand tous les nœuds sont Ready.8. Récupération du kubeconfig via openstack coe cluster config.
Post-succès	Le cluster Kubernetes est opérationnel, accessible via kubectl.

Post-échec	Stack interrompue, log d'erreurs généré, cluster non créé.

UC02 — Déploiement d'une Stack Hybride (VM + Cluster) via Heat

Titre	UC02 — Déployer une stack hybride via Heat
Résumé	Crée simultanément des VMs et un cluster conteneurisé en utilisant un template Heat HOT.
Acteurs	Ingénieur DevOps, Service Heat, Service Magnum
Précondition	Template Heat valide, ressources suffisantes, réseau créé.
Scénario	1. Soumission du template HOT (YAML) définissant OS::Nova::Server et OS::Magnum::Cluster.2. Heat Engine analyse et construit le graphe de dépendances.3. Heat → Neutron : réseau privé, sous-réseau, routeur.4. Heat → Nova : création de la VM base de données (état ACTIVE).5. Heat → Magnum : requête de création du cluster K8s avec le réseau déjà créé.6. Magnum pilote sa stack interne pour déployer les nœuds.7. Heat agrège les statuts → CREATE_COMPLETE quand tout est prêt.8. Les conteneurs K8s peuvent désormais requêter la VM Nova via IP privée.
Post-succès	Stack hybride opérationnelle et fonctionnelle.
Post-échec	Rollback automatique (--rollback-on-failure), toutes ressources supprimées.

UC03 — Mise à l'Échelle d'un Cluster

Titre	UC03 — Mettre à l'échelle un cluster
Résumé	Augmenter ou réduire dynamiquement le nombre de nœuds workers d'un cluster.
Acteurs	Ingénieur DevOps, Service Magnum
Précondition	Cluster déjà opérationnel, ressources disponibles.
Scénario	1. Demande : openstack coe cluster resize <nom> --node-count N.2. Magnum pilote la création ou suppression des VMs workers via Nova.3. Kubernetes détecte automatiquement les nouveaux nœuds et les intègre.4. Rééquilibrage : l'orchestrateur déplace certains pods vers les nouveaux nœuds.
Post-succès	Cluster mis à l'échelle correctement.
Post-échec	Cluster dans un état DEGRADED ou scaling interrompu.

UC04 — Automatisation via Script/API

Titre	UC04 — Automatisation du déploiement de clusters
Résumé	Automatiser la création de clusters via des scripts CLI (Bash, Python, Terraform, Ansible).
Acteurs	Ingénieur DevOps, Service Magnum, Service Heat
Précondition	Scripts disponibles, accès API opérationnel, ClusterTemplate configuré.
Scénario	1. L'ingénieur DevOps exécute un playbook Ansible ou un script Bash/Terraform.2. Le script appelle l'API via openstack coe cluster create avec les paramètres requis.3.

	Magnum contacte Heat pour orchestrer le déploiement de l'infrastructure.4. Nova, Neutron et Cinder provisionnent les ressources.5. Le script récupère automatiquement le fichier kubeconfig une fois le cluster prêt.
Post-succès	Cluster fonctionnel déployé de manière totalement automatique.
Post-échec	Échec avec logs d'erreur, aucune ressource laissée en suspens.

UC05 — Suppression d'un Cluster

Titre	UC05 — Supprimer un cluster
Résumé	Supprimer proprement un cluster Magnum et libérer toutes ses ressources.
Acteurs	Administrateur Cloud, Ingénieur DevOps, Service Magnum
Précondition	Le cluster existe et n'est pas utilisé par un service critique.
Scénario	1. openstack coe cluster delete <nom>.2. Magnum identifie et supprime les composants logiciels (workers, masters).3. Nova procède à la suppression des Machines Virtuelles.4. Heat nettoie les stacks résiduelles (réseaux, volumes).
Post-succès	Cluster totalement supprimé, ressources et quotas libérés.
Post-échec	Cluster dans un état de suppression partielle (ressources verrouillées).

Chapitre 4 — Environnement de Réalisation

4.1 Environnement Logiciel et Technologies

Pour garantir la stabilité et la compatibilité des services OpenStack, l'environnement suivant a été sélectionné :

- **Ubuntu 24.04 LTS (version recommandée pour Kolla-Ansible, gestion robuste des paquets et dépendances Python 3.12)**
- **Kolla-Ansible (méthode de déploiement recommandée en production par la communauté OpenStack, contrairement à DevStack)**
- **VirtualBox pour héberger la VM Ubuntu — KVM (Kernel-based Virtual Machine) utilisé par Nova pour les instances internes**
- **Ansible-core >= 2.15 via environnement virtuel Python /opt/kolla-venv**

4.2 Outils Utilisés

- **Service COE (Container Orchestration Engine). Crée des ClusterTemplates et pilote le cycle de vie des clusters Kubernetes.**
- **Moteur d'orchestration. Interprète les fichiers YAML (HOT) pour déployer atomiquement l'infrastructure (VM, réseaux, stockage).**
- **Interface en ligne de commande pour automatiser les tâches et interagir avec les APIs.**
- **Outil de déploiement OpenStack par conteneurs Docker. Chaque service est isolé dans son propre conteneur.**
- **Langage déclaratif pour définir les infrastructures hybrides (UC02).**
- **Fedora CoreOS pour les nœuds Kubernetes (containerd pré-configuré), Ubuntu 22.04 pour les VMs applicatives.**

4.3 Architecture Logique — Modèle MVC

L'architecture logique d'un service OpenStack (comme Magnum ou Heat) est assimilable au modèle MVC :

- **Les services magnum-api et heat-api reçoivent les requêtes REST, les valident et les transmettent à la logique métier.**
- **La base de données MariaDB stocke l'état des clusters, des stacks et des ressources — source de vérité du système.**
- **Le résultat concret produit : le cluster Kubernetes opérationnel, les VMs créées, ou le tableau de bord Horizon.**

4.4 Architecture Physique — Modèle 3-Tiers

L'architecture physique correspond à un modèle 3-Tiers typique des environnements cloud modernes :

- **Le poste de l'administrateur avec le CLI OpenStack ou le navigateur web (Horizon Dashboard).**

- **Les services de contrôle (Magnum Conductor, Heat Engine, Keystone). C'est ici que les décisions d'orchestration sont prises.**
- **La base de données MariaDB (configurations) et les nœuds de calcul Nova (VMs et conteneurs en exécution réelle).**

Dans notre lab, l'architecture All-in-One de Kolla-Ansible héberge les trois tiers sur une seule VM VirtualBox — configuration suffisante pour valider toutes les fonctionnalités du projet.

Chapitre 5 — Architecture Réseau et Services OpenStack

5.1 Prérequis Matériels (VM VirtualBox)

Composant	Minimum	Recommandé pour ce projet
RAM	12 Go	16 Go (+ swap 8 Go = 24 Go effectif)
CPU	4 vCPUs (VT-x/AMD-V)	8 vCPUs avec virtualisation imbriquée activée
Disque	80 Go	120 Go SSD (les images Docker OpenStack consomment ~20 Go)
Réseau	1 interface NAT	2 interfaces : NAT (internet) + Host-Only (enp0s8, IP 192.168.56.x)
OS	Ubuntu 24.04 LTS	Ubuntu 24.04.4 LTS (version utilisée)

5.2 Architecture Réseau

Réseau	CIDR	Rôle
Management	10.0.0.0/24	Communication interne entre services OpenStack
Tunnel (VXLAN)	10.0.1.0/24	Trafic réseau des VMs via OVN
External/Public	192.168.100.0/24	Floating IPs et accès depuis l'extérieur
Tenant (K8s)	10.11.12.0/24	Réseau privé du cluster Kubernetes
Host-Only (Lab)	192.168.56.0/24	Accès depuis l'hôte VirtualBox (interface enp0s8)

5.3 Flux de Création d'un Cluster Kubernetes

#	Service	Action
1	**CLI → Keystone**	Authentification, obtention d'un token d'accès
2	**CLI → Magnum API**	Envoi de la requête (nom, template, node_count)
3	**Magnum API**	Validation des quotas et du ClusterTemplate
4	**Magnum → RabbitMQ**	Message asynchrone au Magnum Conductor
5	**Magnum Conductor**	Génération dynamique du template Heat HOT
6	**Magnum → Heat API**	Soumission du template YAML à Heat
7	**Heat Engine**	Analyse du graphe de dépendances, création ordonnée
8	**Heat → Nova**	Création des VMs masters/workers via KVM
9	**Heat → Neutron**	Réseau tenant, ports, groupes de sécurité
10	**cloud-init (VM)**	Script d'auto-config : installation de Kubernetes via Ignition/CoreOS
11	**Magnum Conductor**	Polling jusqu'à CREATE_COMPLETE

12	**Magnum**	Génération et stockage du kubeconfig
13	**Utilisateur**	Récupération via : openstack coe cluster config <nom>

Chapitre 6 — Guide de Déploiement Complet

6.1 Séquence d'Installation — Vue d'Ensemble

#	Étape	Description
1	**Mise à jour système**	sudo apt update && apt upgrade ; installation des outils de base
2	**Création du swap (8 Go)**	Obligatoire pour éviter le blocage init_placement par OOM Killer
3	**Installation de Docker**	Dépôt officiel Docker CE ; ajout au groupe docker
4	**Configuration réseau**	Identification des interfaces enp0s3 (NAT) et enp0s8 (Host-Only)
5	**Installation Kolla-Ansible**	Virtualenv Python /opt/kolla-venv ; pip install kolla-ansible
6	**Configuration globals.yml**	network_interface: enp0s8 ; kolla_internal_vip_address: 192.168.56.101
7	**Volume Cinder LVM**	Fichier loopback 10 Go → pvcreate → vgcreate cinder-volumes
8	**Bootstrap des nœuds**	kolla-ansible bootstrap-servers -i ~/all-in-one
9	**Préchecks**	kolla-ansible prechecks -i ~/all-in-one (tous les checks doivent passer)
10	**Déploiement OpenStack**	kolla-ansible deploy -i ~/all-in-one (30–60 min, ne pas interrompre)
11	**Post-déploiement**	kolla-ansible post-deploy ; source /etc/kolla/admin-openrc.sh

6.2 Installation de Kolla-Ansible (Étape 5)

L'installation se fait dans un environnement virtuel Python dédié pour isoler les dépendances :

```
python3 -m venv /opt/kolla-venvsource /opt/kolla-venv/bin/activatepip install -U pippip install 'ansible-core>=2.15,<2.16'pip install kolla-ansibleecho 'source /opt/kolla-venv/bin/activate' >> ~/.bashrc
```

6.3 Configuration globals.yml (Étape 6)

Le fichier /etc/kolla/globals.yml est le cœur de la configuration Kolla-Ansible. Les paramètres critiques sont :

```
kolla_base_distro: 'ubuntu'kolla_install_type: 'source'openstack_release: '2023.2'network_interface: 'enp0s8'        # Interface Host-Onlyneutron_external_interface: 'enp0s3'  # Interface NATkolla_internal_vip_address: '192.168.56.101'  # IP Host-Onlyenable_haproxy: 'no'neutron_plugin_agent: 'ovn'enable_magnum: 'yes'enable_heat: 'yes'enable_cinder: 'yes'enable_cinder_backend_lvm: 'yes'nova_compute_virt_type: 'kvm'
```

6.4 Configuration Post-Déploiement (Étape 10)

Une fois le déploiement terminé (failed=0 dans le PLAY RECAP), les étapes suivantes permettent d'initialiser l'environnement OpenStack :

8. **kolla-ansible post-deploy -i ~/all-in-one** pour la post-configuration automatique
9. **source /etc/kolla/admin-openrc.sh** pour charger les variables d'environnement admin
10. **openstack network create --share --external --provider-physical-network physnet1 --provider-network-type flat public**
11. **openstack flavor create k8s.worker --vcpus 2 --ram 4096 --disk 40**
12. **Obligatoire avec --property os_distro=fedora-coreos pour que Magnum la reconnaisse**
13. **openstack keypair create --public-key ~/.ssh/openstack_k8s_key.pub k8s-keypair**

6.5 Déploiement via Magnum — Cluster Kubernetes (Étapes 16–18)

Création du ClusterTemplate qui définit les paramètres par défaut de tous les clusters :

```
openstack coe cluster template create k8s-template \  --image fedora-coreos-latest \  --keypair
k8s-keypair \  --external-network public \  --flavor k8s.worker \  --master-flavor k8s.master \
--docker-volume-size 50 \  --network-driver calico \  --coe kubernetes \  --labels
kube_tag=v1.26.4,container_runtime=containerd \  --master-lb-enabled --floating-ip-enabled
```

Création du cluster Kubernetes (1 master + 3 workers) :

```
openstack coe cluster create mon-cluster-k8s \  --cluster-template k8s-template \  --master-count 1
--node-count 3 --timeout 60# Surveiller la progression (10–25 min)watch -n 30 'openstack coe cluster
list'
```

Accès au cluster avec kubectl :

```
openstack coe cluster config mon-cluster-k8s --dir ~/.kube --forceexport
KUBECONFIG=~/.kube/configkubectl get nodes# Résultat attendu : 1 master + 3 workers en statut
'Ready'
```

6.6 Orchestration Hybride via Heat (Étapes 21–23)

Le template HOT hybrid-stack.yaml définit en une seule description YAML une infrastructure complète combinant une VM PostgreSQL et un cluster Kubernetes Magnum. Les paramètres clés du template sont :

- **VM base de données PostgreSQL avec user_data cloud-init pour installation et configuration automatique**
- **Réseau privé partagé 10.20.30.0/24 avec accès externe via routeur**
- **Cluster Kubernetes utilisant le template k8s-template, depends_on router_interface**
- **IP flottante pour accès SSH au serveur PostgreSQL**

Déploiement de la stack hybride :

```
# Valider le template avant déploiementopenstack orchestration template validate -t
~/heat-templates/hybrid-stack.yaml# Déployer avec rollback automatique en cas d'échecopenstack
stack create hybrid-infra \  -t ~/heat-templates/hybrid-stack.yaml \  --parameter
keypair_name=k8s-keypair \  --parameter k8s_node_count=2 \  --rollback-on-failure
```

Chapitre 7 — Problèmes Rencontrés et Solutions

Ce chapitre constitue un guide de dépannage complet basé sur les erreurs réellement rencontrées lors du déploiement. Chaque problème est documenté avec sa cause racine et la solution appliquée.

Erreur rencontrée	Cause racine	Solution appliquée
kolla-ansible -i <inv> <cmd> : command not found	Syntaxe inversée (commande après -i)	Utiliser : kolla-ansible <cmd> -i ~/all-in-one
sudo: mot de passe requis (Ansible)	Sudo interactif bloquant Ansible	vboxuser ALL=(ALL) NOPASSWD:ALL dans sudoers
No module named 'docker'	SDK Python Docker absent du venv	pip install docker dans /opt/kolla-venv
No module named 'dbus'	Module dbus absent du venv	apt install python3-dbus + copie dans le venv
Hostname non unique (RabbitMQ)	Plusieurs IPs résolues par le hostname	Corriger /etc/hosts + utiliser IP Host-Only 192.168.56.101
Volume group 'cinder-volumes' not found	Loopback perdu après reboot / espace insuffisant	Recréer loopback 10 Go sur loop libre + rc.local permanent
Disque à 90% (5.8 Go libres)	Fichier cinder 31 Go + snaps Firefox/LibreOffice	Supprimer le fichier loopback surdimensionné + snaps inutiles

7.1 Détail des Corrections Critiques

Problème 1 : Hostname non unique (RabbitMQ precheck)

L'erreur 'Hostname has to resolve uniquely to the IP address of api_interface' est apparue car le hostname ubuntun résolvait vers plusieurs IPs (10.0.2.15 NAT + 172.17.0.1 Docker bridge). La solution a consisté à :

14. Identifier l'IP de l'interface Host-Only (enp0s8) : 192.168.56.101
15. Corriger /etc/hosts pour n'avoir qu'une seule entrée : 192.168.56.101 ubuntun
16. Mettre à jour globals.yml : network_interface: 'enp0s8' et kolla_internal_vip_address: '192.168.56.101'

Problème 2 : Espace disque insuffisant

Avec 90% d'utilisation (48G/57G), le déploiement OpenStack (images Docker ~20 Go) était impossible. Les actions menées :

17. Suppression du fichier loopback surdimensionné (31 Go) de cinder-volumes
18. Suppression des snaps inutiles : Firefox, LibreOffice, gnome, gtk-common-themes, firmware-updater, snap-store (~3.6 Go)
19. Nettoyage des anciens kernels HWE (270 Mo libérés)
20. Recréation d'un volume Cinder de 10 Go suffisant pour le lab

Problème 3 : Volume Cinder (loop device occupé)

Tous les /dev/loop0 à /dev/loop16 étaient utilisés par snapd. La solution : utiliser losetup -f pour trouver automatiquement le premier loop device libre, puis créer le VG sur ce device. La persistance est assurée via /etc/rc.local.

Chapitre 8 — Gestion, Sécurité et Bonnes Pratiques

8.1 Gestion du Cycle de Vie des Clusters

Scaling des Clusters

```
# Scale-up : passer de 3 à 5 nœudsopenstack coe cluster resize mon-cluster-k8s --node-count 5# Scale-down : réduire à 2 nœudsopenstack coe cluster resize mon-cluster-k8s --node-count 2# Kubernetes drainera automatiquement les pods des nœuds supprimésskubectl get nodes
```

Gestion des Quotas

```
openstack quota set \  --cores 32 --ram 65536 --instances 20 \  --volumes 30 --gigabytes 500 --floating-ips 15 \  k8s-project
```

Nettoyage des Ressources Orphelines

Un script de nettoyage automatique a été créé pour libérer les ressources non utilisées (volumes disponibles, IPs flottantes non associées). Ce script s'intègre dans une démarche de gestion des coûts cloud.

8.2 Sécurisation des Accès

- **Règles strictes pour les nœuds K8s — port 6443 (API K8s) uniquement depuis le réseau admin, port 10250 (Kubelet) uniquement depuis le groupe k8s-strict**
- **Rôles différenciés (admin, k8s-developer, k8s-viewer) par projet OpenStack**
- **Paire de clés RSA 4096 bits spécifique au projet (openstack_k8s_key)**
- **Communications chiffrées entre services OpenStack et certificats K8s auto-générés par Magnum**

8.3 Infrastructure as Code et Versionning

- **Tous les templates d'infrastructure dans ~/iac-openstack**
- **Les fichiers de secrets (openrc, passwords, clés) ne sont jamais versionnés**
- **openstack orchestration template validate avant chaque stack create**
- **Option --rollback-on-failure sur tous les déploiements Heat**

8.4 Diagnostic et Commandes de Surveillance

Commandes essentielles de diagnostic quotidien :

```
# Conteneurs Kolladocker ps --format 'table {{.Names}}\t{{.Status}}'# Logs d'un servicedocker logs kolla_magnum_api --tail 100 -f# Cluster K8skubectl get nodeskubectl get pods -Akubectl top nodes# Stack Heatopenstack stack event list <NOM_STACK> --nested-depth 5
```

Chapitre 9 — Conclusion et Perspectives

9.1 Récapitulatif des Réalisations

Ce projet a permis de mettre en place une plateforme cloud complète et fonctionnelle reposant sur OpenStack déployé via Kolla-Ansible. Les réalisations couvrent l'ensemble du cycle défini dans le cahier des charges :

#	Phase	Action clé	Résultat
0	**Préparation**	Swap 8 Go + configuration VirtualBox	RAM suffisante, fin des blocages OOM
1	**Infrastructure**	Ubuntu 24.04 + Docker + réseau dual-interface	Environnement prêt pour Kolla
2	**Kolla-Ansible**	Installation + configuration globals.yml	Outil de déploiement OpenStack prêt
3	**Volume Cinder**	LVM loopback 10 Go avec persistance rc.local	Stockage bloc pour volumes K8s
4	**Déploiement OpenStack**	kolla-ansible deploy (30–60 min)	OpenStack complet dans Docker
5	**Réseau & Images**	Réseau externe + Fedora CoreOS + Ubuntu 22.04	Infrastructure de base prête
6	**Cluster K8s**	ClusterTemplate Magnum + cluster 1 master/3 workers	Kubernetes opérationnel
7	**Stack Hybride**	Template HOT hybrid-stack.yaml	VM+K8s déployés en une commande
8	**Gestion & Sécurité**	Scaling, RBAC, monitoring, Git IaC	Infrastructure en production

9.2 Checklist de Validation Finale

- **Swap de 8 Go actif**
- **Tous les conteneurs en état 'Up'**
- **Tous les services visibles**
- **magnum-api et magnum-conductor en état 'up'**
- **Cluster en statut CREATE_COMPLETE**
- **Tous les nœuds en statut 'Ready'**
- **hybrid-infra en statut CREATE_COMPLETE**
- **Scaling fonctionnel**

9.3 Apports Pédagogiques

Ce projet a été l'occasion d'approfondir de nombreuses compétences techniques et méthodologiques :

- **Déploiement OpenStack complet par conteneurs Docker sur Ubuntu 24.04**

- **Architecture multi-service, flux de création de ressources, APIs REST**
- **Déploiement via Magnum, gestion du cycle de vie, kubeconfig**
- **Templates HOT Heat, versionning Git, rollback automatique**
- **Diagnostic système, gestion des dépendances Python, LVM loopback**

9.4 Perspectives

Ce travail constitue une base solide pour des développements futurs, notamment :

- Mise en place d'un monitoring complet avec Prometheus + Grafana pour la supervision des métriques CPU/RAM/réseau des clusters
- Intégration de pipelines CI/CD (GitLab CI, Jenkins) déclenchant des déploiements automatiques sur les clusters Kubernetes
- Déploiement multi-nœuds Kolla-Ansible avec séparation des rôles (contrôleur, calcul, stockage) pour une architecture haute disponibilité
- Évaluation de solutions de stockage distribué (Ceph) pour remplacer le backend LVM loopback par une solution production-ready
- Mise en place d'Octavia (Load Balancer as a Service) pour les services Kubernetes de type LoadBalancer

Ce rapport constitue la documentation complète du projet de conteneurisation et gestion des workloads avec OpenStack Magnum & Kubernetes, réalisé à l'Université Saint Jean sous la supervision de M. NGUIMBUS Emmanuel, Année Académique 2025–2026.

ESSONO EFFA Etienne Florian & NGNINTEDEM FEUPI Vaillant
Université Saint Jean — Saint Jean Ingénieur — 2025–2026